



Clase 24 -

Agenda:



9:00 - 9:50 → Herramientas de Visualización de Datos y Ecosistema Big Data

9:50 - 11:20 → Ejercicios

11:20 - 11:40 → Descanso

11:40 - 12:40 → Actividad: Demostraciones de algunas herramientas open source de Visualización para Big Data

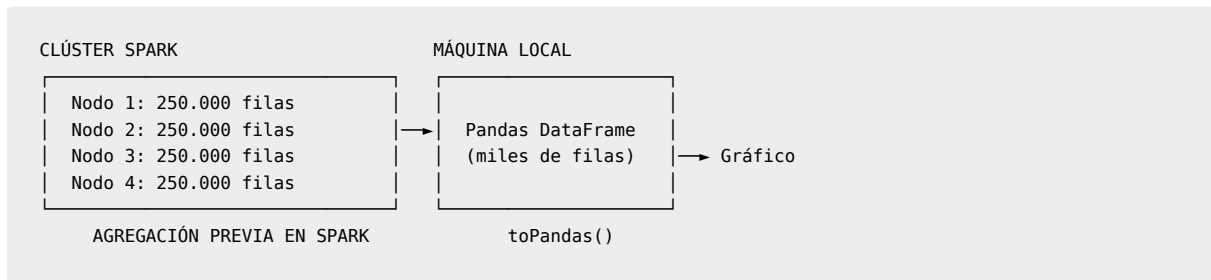
12:40 - 14:00 → Actividad: Demostraciones de algunas herramientas open source de Visualización para Big Data

Sesión 1: Herramientas de Visualización de Datos y Ecosistema Big Data

1. El problema de visualizar Big Data

Spark procesa millones de filas distribuidas en varios nodos. Las bibliotecas de visualización como Matplotlib o Seaborn trabajan en local, en la memoria de **un solo ordenador**. Estas dos realidades no se hablan directamente.

Imagina que tienes un almacén con un millón de cajas repartidas en diez naves. Para enseñarle el contenido a alguien, no puedes traer las diez naves a tu casa: **primero resumes, luego transportas**.



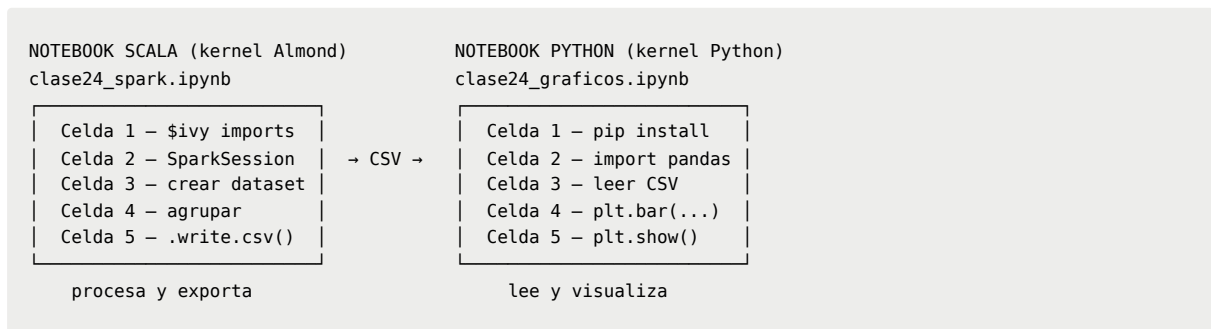
La regla de oro:

Agrupas y reduces en Spark → exporta el resumen a CSV → dibuja en local. Nunca hagas `toPandas()` sobre un DataFrame de millones de filas sin filtrar antes. El driver se quedará sin memoria.

2. El puente: dos notebooks

⚠ **Diferencia importante con Databricks:** En Databricks o Apache Zeppelin puedes mezclar celdas Scala y Python en el mismo notebook. En nuestro entorno (**Almond + VSCode**) eso **no es posible**: cada notebook tiene un único kernel. Si intentas ejecutar código Python en una celda Almond, obtendrás un error de compilación.

La solución en este curso: usamos **dos notebooks separados**.



El CSV en disco es el "mensajero" entre los dos notebooks. `coalesce(1)` fuerza que Spark escriba **un único archivo** en la carpeta de salida — sin esto, crearía tantos archivos `part-NNNNN.csv` como particiones tenga el DataFrame.

3. Matplotlib y Seaborn: gráficos desde Spark

Matplotlib es la biblioteca de visualización más usada en Python: control total sobre cada elemento del gráfico. Todo gráfico sigue cinco pasos: importar las bibliotecas, leer el CSV con `glob`, crear la figura con `plt.subplots()`, dibujar con el método correspondiente y mostrar con `plt.show()`.

Gráfico	Método	Cuándo usarlo
Barras verticales	<code>plt.bar()</code>	Comparar categorías
Barras horizontales	<code>plt.barh()</code>	Rankings con etiquetas largas

Gráfico	Método	Cuándo usarlo
Líneas	<code>plt.plot()</code>	Evolución temporal
Histograma	<code>plt.hist()</code>	Distribución de una variable numérica
Dispersión	<code>plt.scatter()</code>	Relación entre dos variables

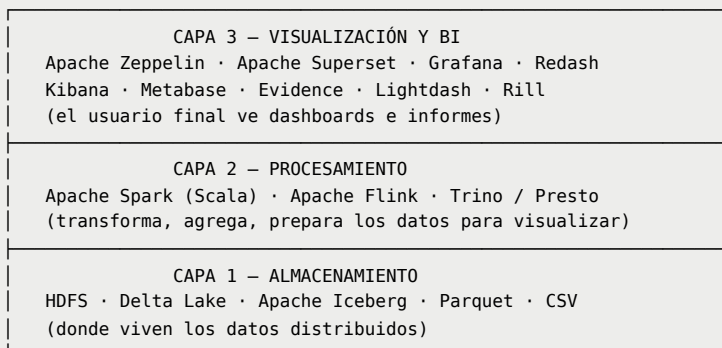
Seaborn está construido sobre Matplotlib y añade gráficos estadísticos con menos código.

`sns.heatmap()` es ideal para matrices de correlación; `sns.boxplot()` muestra mediana, cuartiles y valores atípicos agrupados por categoría. Los códigos completos están en `clase24_graficos.ipynb`.

4. Ecosistema open source de visualización Big Data

4.1 Mapa del ecosistema: tres capas

Las herramientas open source que un ingeniero de datos con Scala/Spark encontrará en su carrera se organizan en tres capas:



Scala + Spark vive en la **capa 2**. Su trabajo es alimentar a las herramientas de la **capa 3**, que son las que los analistas de negocio y directivos usan sin escribir código.

4.2 Apache Zeppelin — El notebook nativo de Big Data

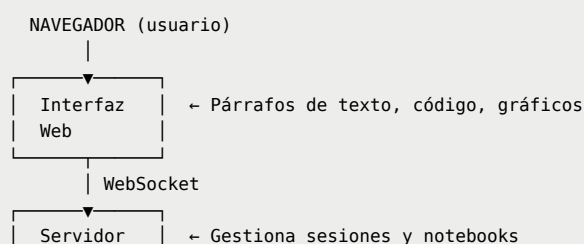
Repositorio: <https://github.com/apache/zeppelin> **Licencia:** Apache 2.0

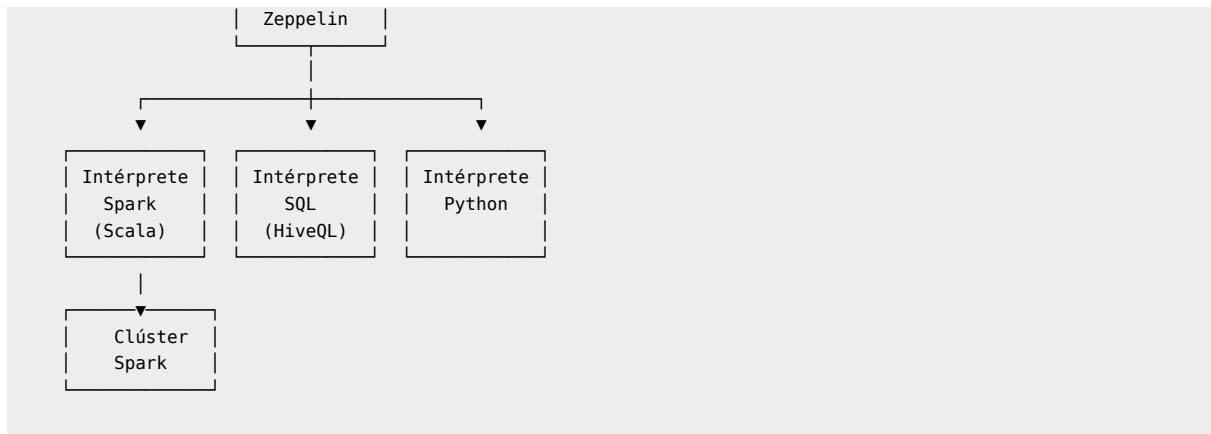
Lenguajes soportados: Scala, SQL, Python, R, Markdown, Shell

¿Qué es?

Apache Zeppelin es un notebook web interactivo diseñado específicamente para el ecosistema Big Data. A diferencia de Jupyter, fue construido desde el principio con Spark en mente: el intérprete de Spark (Scala) está integrado de serie sin necesidad de kernels externos.

Arquitectura interna





La gran ventaja: gráficos sin código adicional

Cuando ejecutas una query SQL o Scala que devuelve un DataFrame, aparece automáticamente una barra de iconos con la que puedes cambiar entre tabla, barras, líneas, dispersión y tarta con **un solo clic**.

```
// En un párrafo Zeppelin con intérprete %spark
val ventas = spark.read.parquet("/datos/ventas")
  .groupBy("categoria")
  .agg(sum("importe").alias("total"))
  .orderBy(desc("total"))

z.show(ventas) // ← muestra la tabla con controles de gráfico integrados
```

Debajo del resultado aparece: [Tabla] [Barras] [Líneas] [Dispersión] [Tarta]

Cuándo usar Zeppelin vs Jupyter + Almond

Criterio	Zeppelin	Jupyter + Almond
Gráficos sin código	✓ Integrados	✗ Necesita Python/Matplotlib
Soporte Scala nativo	✓ De serie	✓ Con kernel Almond
Colaboración multiusuario	✓ Sí	⚠ Con JupyterHub
Instalación local (Windows)	⚠ Más compleja	✓ Sencilla con Almond
Ideal para	Clústeres Spark en empresa	Desarrollo local y aprendizaje

4.3 Apache Superset — El BI open source de referencia

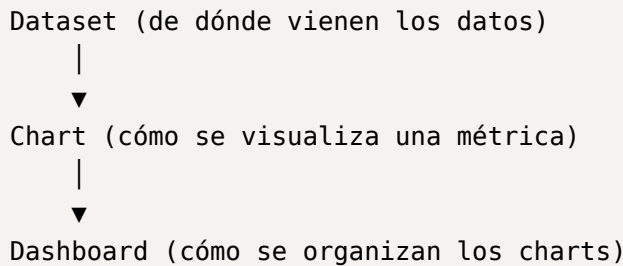
Repositorio: <https://github.com/apache/superset> **Licencia:** Apache 2.0

GitHub Stars: +70.000 ★

Usado por: Airbnb, Dropbox, Lyft, Twitter, Nielsen

¿Qué es?

Apache Superset es una plataforma web de Business Intelligence creada por Airbnb en 2015 y donada a la Apache Software Foundation en 2017. Su filosofía central es la separación en tres capas:



Arquitectura de conexión con Spark

Superset —JDBC→ Spark Thrift Server → Spark SQL → Parquet/Delta

El patrón más común en producción usa una base de datos intermedia:

Spark (ETL) → ClickHouse → Superset

Esto desacopla el procesamiento de la visualización y evita que las queries de Superset compitan con los jobs de Spark.

Tipos de gráfico disponibles (+40)

Básicos	Avanzados	Especializados
Barras	Treemap	Mapa de calor (geo)
Líneas	Sunburst	Scatterplot animado
Área	Sankey (flujos)	Word cloud
Tarta	Chord diagram	Histograma 2D
Tabla	Box plot	Funnel (embudo)
KPI (Big Number)	Waterfall	Pivot table

Las cuatro áreas de la interfaz

SQL Lab — editor SQL completo con autocompletado, historial y posibilidad de guardar resultados como datasets.

Datasets — capa de abstracción donde se definen métricas calculadas una sola vez: `SUM(importe) AS ventas_totales`. Esta es la clave de la consistencia.

Charts — cada chart es dataset + tipo de visualización + configuración. Se reutiliza en múltiples dashboards.

Dashboards — colección de charts con layout libre y filtros globales. Un único selector de "Ciudad" puede filtrar simultáneamente diez charts de distintos datasets.

Lo que Superset hace mejor que sus competidores

- El editor **SQL Lab** es más completo que el de Metabase o Grafana
- La **capa semántica de datasets** garantiza que todos calculan las métricas igual
- Más de **40 tipos de gráfico** incluyendo opciones que otros BI open source no tienen

Lo que no hace bien

- La instalación requiere ~6 contenedores Docker (Redis, Celery, PostgreSQL, etc.)
- No optimiza las queries: las ejecuta tal cual contra la fuente de datos
- La curva de aprendizaje del modelo mental (dataset → chart → dashboard) no es inmediata

4.4 Grafana — Plataforma de dashboards y observabilidad

Repositorio: <https://github.com/grafana/grafana> **Licencia:** AGPLv3

Especialidad: dashboards multipanel interactivos, series temporales, alertas

💡 **Aclaración importante:** Grafana **sí permite crear dashboards estilo cuadro de mando** completos, con múltiples paneles, filtros globales, variables interactivas y alertas automáticas. Su fama de "herramienta solo para métricas" viene de su origen, pero lleva años siendo usada como plataforma BI general.

¿Qué es un dashboard en Grafana?

DASHBOARD "VENTAS" [Ciudad: Madrid ▼] [Mes ▼]	
Stat (KPI grande) Total: 9.200 €	Gráfico de barras Ventas por categoría
Gráfico de líneas Evolución mensual	Tabla interactiva Top productos

Al cambiar "Ciudad" en el desplegable, TODOS los paneles se actualizan

Tipos de paneles disponibles (~30)

VISUALIZACIONES	INDICADORES	ESPECIALES
Gráfico de líneas	Stat (nº grande / KPI)	Mapa geoespacial
Barras (vert/horiz)	Gauge (velocímetro)	Heatmap
Área	Bar gauge	Candlestick (finanzas)
Scatter plot	State timeline	Logs viewer
Tarta / Donut	Status history	Texto (Markdown)
Tabla interactiva	Histogram	Canvas (libre)

Variables interactivas: el corazón del cuadro de mando

Las variables convierten un conjunto de gráficos en un verdadero cuadro de mando interactivo:

- Query que alimenta la variable \$ciudad en Grafana `SELECT DISTINCT ciudad FROM kpi_ciudad_mes ORDER BY ciudad`
- Los paneles usan la variable: `SELECT mes, total_ventas FROM kpi_ciudad_mes WHERE ciudad = '$ciudad' ← se sustituye por el valor del desplegable ORDER BY mes`

Hay cinco tipos de variable: **Query** (valores desde BD), **Custom** (valores fijos), **Constant**, **Interval** (granularidad temporal) y **Ad hoc filters** (el más potente: filtra por cualquier columna sin

configuración previa).

Las variables se encadenan: el valor de `$ciudad` puede alimentar la query que genera los valores de `$tienda`.

Alertas integradas en los paneles

```
Condición: "si ventas_hoy < (media_7_días × 0.70)" → caída >30%
|
▼ Grafana evalúa la query cada 5 minutos
|
¿Se cumple?
SÍ → Notificación a Slack / Teams / Email / PagerDuty
NO → (sigue monitorizando en silencio)
```

El sistema de alertas incluye **Contact points** (Slack, email, PagerDuty, Teams, Telegram), **Notification policies** (enrutado y frecuencia) y **Silences** (pausas para mantenimientos).

Conexión con Spark

```
Spark (Scala) —escribe→ PostgreSQL / MySQL    —lee→ Grafana
Spark (Scala) —escribe→ ClickHouse              —lee→ Grafana
Spark (Scala) —escribe→ Apache Druid             —lee→ Grafana
Spark (Scala) —escribe→ Elasticsearch            —lee→ Grafana
```

Grafana soporta más de 150 fuentes de datos mediante plugins.

Lo que Grafana no hace bien

- **No tiene capa semántica.** Cada panel tiene su propia query. Si cambias la definición de una métrica, hay que actualizar cada panel individualmente.
- **No tiene explorador de datos.** Partes de la query en blanco. No es para exploración, es para quien ya sabe qué quiere ver.
- **Row-Level Security limitado.** Para filtrar filas por usuario hay que implementarlo en la base de datos.

Comparativa Grafana vs Superset

Criterio	Grafana	Apache Superset
Dashboards multipanel	✓ Completo	✓ Completo
Variables / filtros interactivos	✓ Muy potente	✓ Bueno
Series temporales	✓ Excelente	⚠ Básico
Dashboards de negocio clásico	✓ Bueno	✓ Excelente
Alertas y notificaciones	✓ Integradas y potentes	⚠ Limitadas
SQL ad-hoc para el analista	⚠ Básico	✓ Editor SQL completo
Capa semántica	✗ No tiene	✓ Datasets reutilizables
Ideal para	Ops + negocio + alertas	Analítica de negocio pura

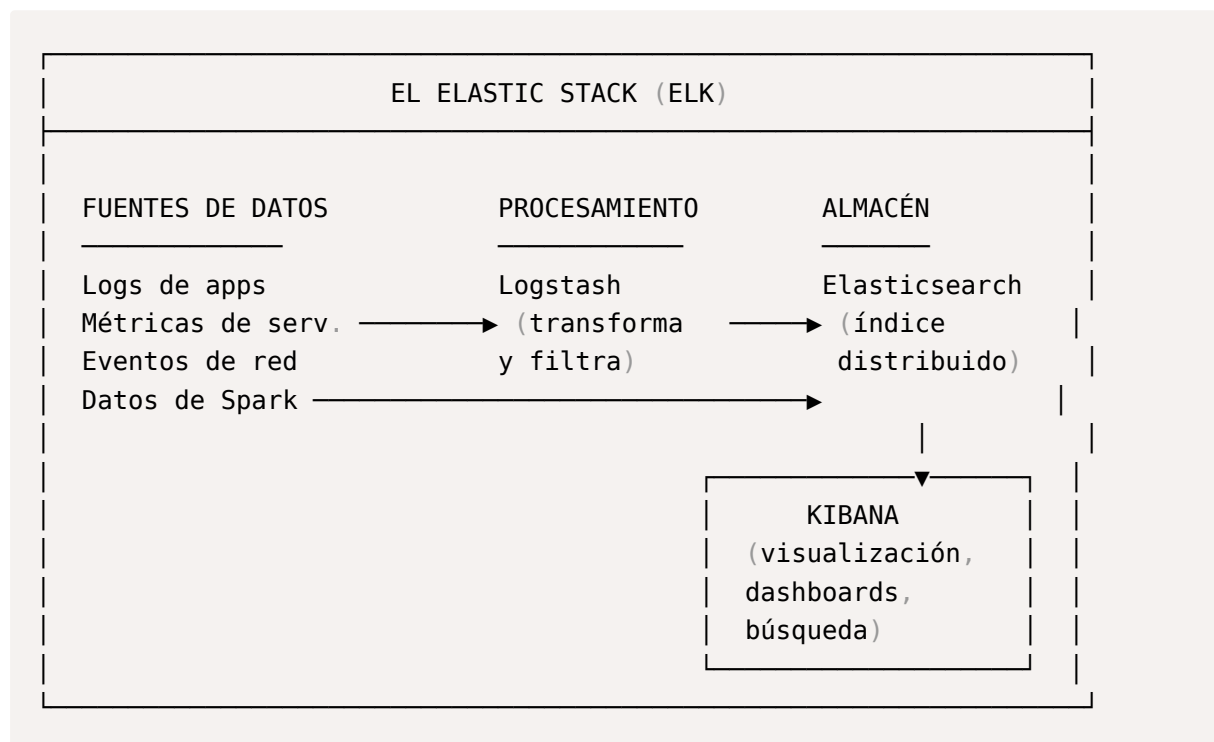
4.5 Kibana y el Elastic Stack (ELK)

Repositorio: <https://github.com/elastic/kibana> **Licencia:** Elastic License 2.0 (open source con restricciones comerciales desde 2021)

Especialidad: búsqueda y análisis de logs, eventos y series temporales; dashboards de observabilidad

⚠ **Nota sobre la licencia:** En 2021 Elastic cambió la licencia de Apache 2.0 a Elastic License 2.0. Para uso interno en empresa o aprendizaje es completamente libre. Para uso en servicios cloud competidores hay restricciones. La alternativa 100% Apache 2.0 es **OpenSearch + OpenSearch Dashboards** (fork de Amazon).

El Elastic Stack completo: ELK



Elasticsearch vs SQL: equivalencias

SQL (Spark / base de datos)		Elasticsearch
Tabla	→	Índice
Fila	→	Documento (JSON)
Columna	→	Campo
SELECT ... GROUP BY	→	Aggregation (terms, sum, avg...)
WHERE	→	Query (match, range, bool...)

Cómo conectar Spark (Scala) con Elasticsearch

```
import $ivy.`org.elasticsearch::elasticsearch-spark-30_2.13:8.13.0`
import org.elasticsearch.spark.sql._

val spark = SparkSession.builder()
    .master("local[*]")
```

```

.appName("Spark-Kibana")
.config("es.nodes", "localhost")
.config("es.port", "9200")
.config("es.index.auto.create", "true")
.getOrCreate()

// Agregar en Spark y escribir el resultado en Elasticsearch
val ventasPorCategoria = ventasConImporte
  .groupBy("categoria", "ciudad")
  .agg(
    sum("importe").alias("total_ventas"),
    count("*").alias("num_transacciones")
  )

// Guardar en el índice "ventas-resumen" de Elasticsearch
ventasPorCategoria
  .write
  .format("es")
  .mode("overwrite")
  .save("ventas-resumen/doc")

println("✅ Datos enviados a Elasticsearch")

```

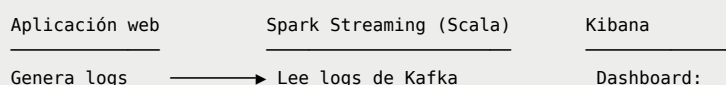
Tipos de visualización en Kibana

Tipo	Descripción
Lens	Editor visual drag & drop para cualquier tipo de gráfico
Discover	Busca un evento concreto en millones de registros en milisegundos
TSVB	Gráficos de series temporales avanzados con múltiples métricas
Maps	Mapas geoespaciales con datos de latitud/longitud
Canvas	Informes visuales estilo presentación con datos en tiempo real
Dashboard	Combina múltiples visualizaciones con filtros globales

El flujo completo Spark → Elasticsearch → Kibana

1. Spark (Scala) lee datos crudos (CSV, Parquet, Kafka...)
2. Spark agrega, transforma y enriquece los datos
3. Spark escribe el resultado en Elasticsearch usando el conector elasticsearch-spark
4. Kibana lee automáticamente el índice de Elasticsearch
5. El analista crea visualizaciones arrastrando campos
6. El dashboard se actualiza en tiempo real cada vez que Spark escribe nuevos datos

Caso de uso real: pipeline de logs con Spark + Kibana



(errores, peticiones, latencias) Kafka ▶ Parsea y estructura
 ▶ Calcula métricas ¿Cuántos 500?
 ▶ Escribe en ES ¿Latencia media?
 → ¿Qué endpoint falla?

Kibana vs OpenSearch Dashboards

Criterio	Kibana	OpenSearch Dashboards
Licencia	Elastic License 2.0	Apache 2.0 (100% libre)
Funcionalidad	Idéntica en lo esencial	Idéntica en lo esencial
Backend requerido	Elasticsearch	OpenSearch
Uso interno empresa	✅ Libre	✅ Libre

Cuándo elegir Kibana

- ✅ Datos principalmente de logs, eventos y trazas (observabilidad)
- ✅ Necesitas búsqueda de texto libre sobre millones de documentos
- ✅ Caso de uso en tiempo real con actualizaciones frecuentes
- ✅ Monitorización de la propia infraestructura Spark
- ❌ Datos principalmente tabulares y relacionales → mejor Superset
- ❌ El equipo no quiere gestionar un clúster Elasticsearch adicional

4.6 Redash — BI ligero para equipos pequeños

Repositorio: <https://github.com/getredash/redash> **Licencia:** BSD 2-Clause

Especialidad: consultas SQL + dashboards simples + compartir resultados

Conexión con Spark

Redash —SQL→ Spark Thrift Server → Spark SQL → Parquet / Delta

Flujo de trabajo

1. Escribes una query SQL en el editor de Redash
2. Redash la ejecuta contra Spark SQL
3. El resultado aparece como tabla
4. Con un clic conviertes la tabla en gráfico
5. Añades el gráfico a un dashboard
6. Compartes el enlace con tu equipo

Cuándo elegir Redash sobre Superset

- Equipo pequeño (2-10 personas)
- Queries principalmente SQL estándar
- Instalación rápida sin complicaciones
- No se necesitan tipos de gráfico avanzados

4.7 Vegas — Visualización nativa en Scala

Repositorio: <https://github.com/vegas-viz/Vegas> **Licencia:** MIT

Integración: funciona dentro de Jupyter + Almond

¿Qué es Vegas?

Vegas es una biblioteca de visualización declarativa para Scala, basada en la especificación **Vega-Lite**. Permite crear gráficos directamente desde Scala sin saltar a Python.

```
import $ivy.`org.vegas-viz::vegas:0.3.11`
import vegas._
import vegas.render.WindowRenderer._
import vegas.sparkExt._

val datos = Seq(
  Map("categoria" -> "Tecnología", "ventas" -> 18820),
  Map("categoria" -> "Deporte",      "ventas" -> 3545),
  Map("categoria" -> "Hogar",        "ventas" -> 2430)
)

Vegas("Ventas por Categoría")
  .withData(datos)
  .encodeX("categoria", Nom)
  .encodeY("ventas", Quant)
  .mark(Bar)
  .show
```

Limitación importante

⚠ Vegas **NO es una plataforma de dashboards**. Genera un gráfico por celda de Jupyter. No tiene layout multipanel, filtros interactivos ni variables. Es una biblioteca de *rendering* de gráficos individuales, no una herramienta de BI.

👉 **Analogía:** Vegas es como tener una buena cámara de fotos — hace imágenes individuales excelentes. Grafana o Superset son como una sala de exposiciones — organizan y presentan muchas imágenes juntas de forma interactiva.

Para un cuadro de mando de verdad desde Almond, las opciones reales son:

- Exportar a HTML con múltiples gráficos SVG incrustados (manual)
- Usar el notebook Python con Matplotlib `subplots`
- Conectar Grafana o Superset a los datos que Spark exporta

4.8 Delta Lake + Apache Superset: el combo más usado en producción

Delta Lake es una capa de almacenamiento open source que añade transacciones ACID sobre ficheros Parquet.

```
// Escribir en Delta Lake desde Scala/Spark
ventasAgregadas.write
  .format("delta")
```



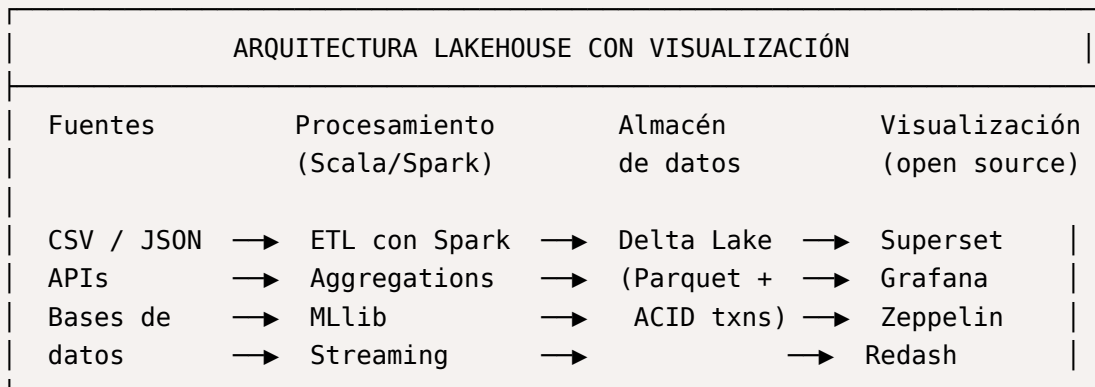
```

.mode("overwrite")
.save("C:/Curso-Scala/datos/delta/ventas_resumen")

// Leer desde Delta Lake
val df = spark.read
  .format("delta")
  .load("C:/Curso-Scala/datos/delta/ventas_resumen")

```

La arquitectura Lakehouse completa



4.9 Panorama ampliado: más herramientas open source

El ecosistema es más amplio que las herramientas ya descritas. Estas son las más relevantes para un Data Engineer con Spark en 2025:

Metabase — El BI más fácil de usar

Repositorio: <https://github.com/metabase/metabase> · **Licencia:** AGPL v3 · **Stars:** ~40.000 ★

El BI open source más instalado del mundo (+60.000 organizaciones). Su filosofía es que cualquier persona de la empresa pueda crear un dashboard sin saber SQL. El constructor visual de queries permite arrastrar columnas y aplicar filtros con clics.

🧐 **Analogía:** Si Superset es el restaurante con carta extensa, Metabase es el menú del día: cuatro opciones claras, llegas, señalas y en cinco minutos tienes tu plato.

Instalación mínima:

bash

```

docker run -d -p 3000:3000 --name metabase metabase/metabase
# Acceder en: http://localhost:3000

```

Apache Druid — Base de datos OLAP con visualización integrada

Repositorio: <https://github.com/apache/druid> · **Licencia:** Apache 2.0

Usado por: Netflix, Airbnb, Twitter, Pinterest

Druid es una base de datos analítica distribuida para consultas de baja latencia sobre datos de alta cardinalidad y en tiempo real. No es solo un frontend: es él mismo la base de datos, con su propia consola web integrada (**Druid Console**).

```
Spark (batch) —Parquet→ Druid → Druid Console (exploración técnica)
Kafka (stream) —————→ Druid → Superset / Grafana (dashboards)
```

Situación	PostgreSQL + Superset	Druid + Superset
1 millón de filas	✅ Perfecto	Innecesariamente complejo
100 millones de filas	⚠️ Lento	✅ Diseñado para esto
Datos en tiempo real	❌	✅ Ingestión nativa desde Kafka
Queries en < 1 segundo	⚠️	✅ Latencia sub-segundo

Evidence — BI como código (SQL + Markdown)

Repositorio: <https://github.com/evidence-dev/evidence> · **Licencia:** MIT

Evidence trata los informes como **código fuente**: combinas queries SQL y texto Markdown en ficheros `.md`, y Evidence los compila en un sitio web estático con gráficos interactivos. Los informes viven en Git, se revisan en Pull Requests.

```
```sql ventas_categoria
SELECT categoria, SUM(importe) AS total
FROM ventas_resumen
GROUP BY categoria
ORDER BY total DESC
```

<BarChart data={ventas_categoria} x="categoria" y="total" />
```

Rill — BI exploratorio ultrarrápido

Repositorio: <https://github.com/rilldata/rill> · **Licencia:** Apache 2.0

Dashboards con respuesta en milisegundos sobre cientos de millones de filas vía DuckDB / ClickHouse. No hay que configurar gráficos: basta señalar una dimensión y una métrica para que Rill genere automáticamente las vistas relevantes.

Streamlit — Aplicaciones de datos interactivas

Repositorio: <https://github.com/streamlit/streamlit> · **Licencia:** Apache 2.0

Permite construir aplicaciones web interactivas de datos escribiendo únicamente Python. Ideal cuando ningún BI drag-and-drop es suficiente y se necesita lógica personalizada.

```
import streamlit as st
import pandas as pd

st.title("Dashboard de Ventas")
df = pd.read_csv("ventas_por_categoria.csv")
ciudad = st.selectbox("Ciudad", df["ciudad"].unique())
```

```
st.bar_chart(df[df["ciudad"] == ciudad].set_index("categoria")["total_ventas"])
```

Lightdash — BI nativo para dbt

Repositorio: <https://github.com/lightdash/lightdash> · **Licencia:** MIT

BI open source construido para equipos que usan **dbt**. Las métricas se definen una sola vez en los modelos dbt y Lightdash las lee automáticamente. Resuelve el problema de que el BI calcula las métricas de forma distinta a como las define el equipo de datos.

4.10 Tabla comparativa global

Herramienta	Tipo	Curva aprendizaje	Conexión Spark	Dashboards	Licencia
Apache Zeppelin	Notebook Big Data	★★	✓ Nativa	⚠ Básico	Apache 2.0
Apache Superset	BI avanzado	★★	✓ Thrift Server	✓ Completo	Apache 2.0
Grafana	Dashboards + obs.	★	✓ JDBC	✓ Completo	AGPLv3
Kibana	Logs + search	★★	✓ elasticsearch-spark	✓ Completo	Elastic Lic.
Redash	BI SQL-first	★	✓ Thrift Server	✓ Completo	BSD 2
Metabase	BI self-service	★ muy baja	✓ JDBC	✓ Completo	AGPL v3
Apache Druid	OLAP DB + UI	★★★★	✓ Ingestión directa	⚠ Básico (usa Superset)	Apache 2.0
Evidence	Code-first BI	★★	✓ DB intermedia	✓ Estático	MIT
Rill	BI exploratorio	★	✓ Parquet/ClickHouse	✓ Operacional	Apache 2.0
Streamlit	App de datos	★★	✓ pandas/PySpark	✓ Custom	Apache 2.0
Lightdash	BI moderno dbt	★★	✓ dbt + warehouse	✓ Completo	MIT
Vegas (Scala)	Biblioteca gráficos	★★	✓ DataFrame Spark	✗ Un gráfico/celda	MIT

4.11 Árbol de decisión: ¿qué herramienta elegir?

¿CUÁL HERRAMIENTA ELEGIR?

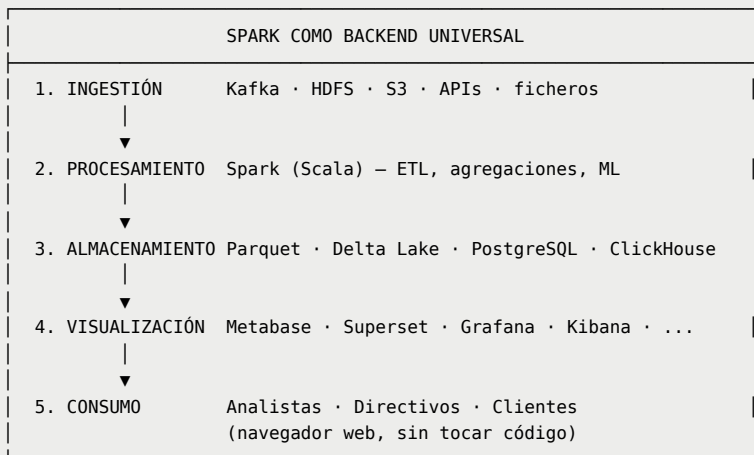
¿Quieres un gráfico individual sin salir de Scala / en el notebook?

SÍ → Vegas (en Almond) – un gráfico por celda
 Zeppelin (en clúster) – gráfico con un clic
 NO

¿Necesitas un cuadro de mando interactivo (dashboard multipanel)?



4.12 El flujo universal: Spark como backend



El Data Engineer con Scala/Spark vive en los pasos 1–3. Su trabajo termina cuando los datos están limpios, agregados y almacenados eficientemente. A partir de ahí, la herramienta de visualización hace el resto.

5. El mundo BI empresarial: qué usan los analistas

5.1 Las herramientas que usan los equipos de BI y analistas de negocio

Los equipos de BI y analistas en empresas medianas y grandes **no suelen usar Superset ni Grafana**. Esas herramientas las usan los ingenieros de datos. Los analistas viven en un ecosistema diferente, dominado por herramientas comerciales con interfaces más pulidas.

Mundo Data Engineering

Mundo BI / Analítica de negocio

Superset	Power BI	← líder en España y Europa
Grafana	Tableau	← referente histórico
Redash	Looker	← integrado con Google Cloud
Metabase	Qlik Sense	← muy fuerte en industria
	MicroStrategy	
	SAP BusinessObjects	
	IBM Cognos	

Power BI (Microsoft) es el líder absoluto en España y Europa. Domina por razones prácticas: está integrado en el ecosistema Microsoft (Office 365, Azure, Excel, Teams), el precio es razonable para empresas con licencias Microsoft, y hay una enorme cantidad de profesionales certificados.

Tableau (Salesforce) es el referente histórico en visualización de alta calidad. Sigue siendo la herramienta preferida en empresas con analistas especializados que priorizan la calidad visual y la potencia de exploración.

Looker (Google) tiene dos productos: Looker enterprise (con su lenguaje LookML) y Looker Studio (antes Google Data Studio), la versión gratuita muy usada para dashboards conectados a Google Analytics y BigQuery.

Qlik Sense tiene una base instalada enorme en Europa, especialmente en empresas industriales alemanas, escandinavas y del norte de Europa.

Cómo se conectan con Spark

Todas estas herramientas consumen los datos que Spark produce, exactamente igual que las open source:

```

Spark (Scala) —procesa—> Data Warehouse (Snowflake / BigQuery / PostgreSQL)
                        |
                        v
                Power BI / Tableau / Looker / Qlik
                conectan y el analista arrastra columnas

```

La división de roles en la práctica

```

Data Engineer (tú, con Scala/Spark)
→ construye el pipeline
→ transforma y limpia los datos
→ escribe resultados en el data warehouse
→ NO toca Power BI ni Tableau

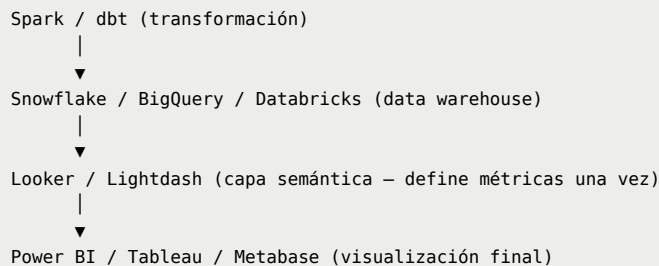
BI Developer / Data Analyst
→ coge los datos del data warehouse
→ construye los modelos semánticos en Power BI / Tableau
→ crea los dashboards para negocio
→ NO toca Spark ni Scala

```

Por qué los analistas no usan Superset ni Grafana

Power BI tiene soporte oficial de Microsoft, certificaciones reconocidas (PL-300), miles de tutoriales, comunidad enorme en español, y si algo falla hay soporte empresarial. Un analista de negocio no quiere gestionar un servidor Docker con Superset ni actualizar versiones Python. Quiere instalar un ejecutable, conectar a la base de datos y ponerse a trabajar.

El Modern Data Stack: la tendencia actual



5.2 Superset en profundidad

Superset nació en Airbnb en 2015 y hoy lo usan en producción Airbnb, Dropbox, Lyft y Twitter. Su modelo mental central es la separación dataset → chart → dashboard, que permite reutilizar el mismo chart en múltiples dashboards y actualizar la fuente de datos sin rehacer las visualizaciones.

Patrón 1 — Spark Thrift Server (más directo): Spark expone un servidor SQL. Superset lo consulta como una base de datos normal. Inconveniente: latencia alta porque Spark arranca jobs para responder.

Patrón 2 — Base de datos intermedia (más común en producción): Spark agrega y escribe en PostgreSQL o ClickHouse. Superset consulta esa BD en milisegundos.

Lo que hace mejor: SQL Lab completo, capa semántica de datasets (una métrica definida una vez, reutilizada en cien charts), +40 tipos de gráfico incluyendo Sunburst, Sankey y mapas geoespaciales con Deck.gl.

Lo que no hace bien: instalación compleja (~6 contenedores Docker), no optimiza queries, no tiene alertas potentes.

5.3 Grafana en profundidad

La confusión más común: mucha gente conoce Grafana por dashboards de infraestructura (CPUs, memoria, red) y cree que "es solo para devops". La realidad es que es una plataforma de dashboards de propósito general que puede visualizar cualquier dato en cualquier base de datos.

La distinción clave:

Grafana = plataforma de dashboards
Prometheus / InfluxDB = fuentes de datos especializadas
Son cosas distintas. Grafana funciona perfectamente con PostgreSQL.

Lo que hace mejor: variables interactivas encadenadas, alertas nativas muy maduras (reglas + contact points + notification policies + silences), más de 150 fuentes de datos vía plugins, curva de aprendizaje baja.

Lo que no hace bien: no tiene capa semántica, no tiene explorador de datos ad-hoc, Row-Level Security limitado.

Grafana Loki: logs sin Elasticsearch

Loki es la alternativa open source a Elasticsearch para logs. Es más barata en almacenamiento porque no indexa el contenido de los logs, solo sus etiquetas.

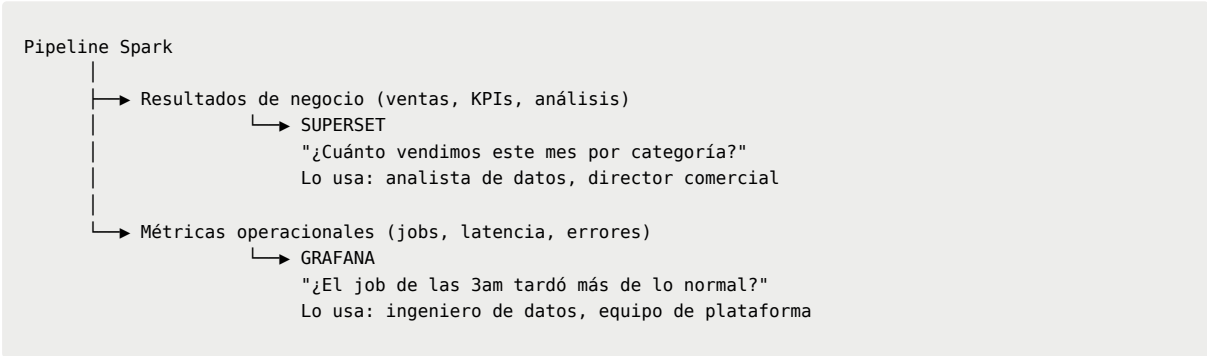
PLG Stack: Promtail (recoge) → Loki (almacena) → Grafana (visualiza)

5.4 Superset vs Grafana: ¿cuál se usa más con Spark?

La respuesta es que **no compiten directamente**: resuelven necesidades distintas dentro del mismo proyecto.

Equipo de Data Engineering / Analytics → Superset
Equipo de DevOps / Platform Engineering → Grafana
Empresa que necesita los dos → los dos, para cosas distintas

En una empresa mediana típica:



La pregunta que lo decide:

- ¿Quién va a usar el dashboard?
- Lo usa el equipo de ingeniería / operaciones → **Grafana**
 - Lo usa el equipo de negocio / analítica → **Superset o Metabase**

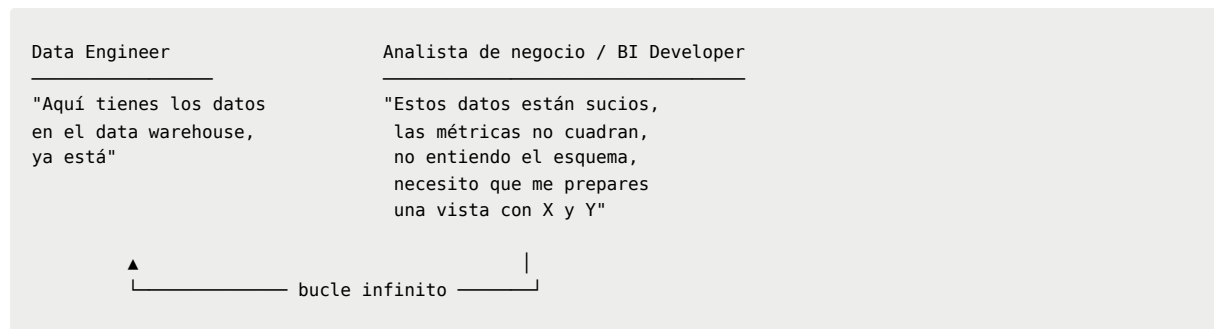
6. Roles en un equipo de datos moderno

6.1 El mapa completo de roles en un equipo de datos moderno

STACK DE DATOS Y SUS ROLES				
FUENTES DE DATOS	DATA ENGINEER (Scala/Spark)	ANALYTICS ENG. (dbt + SQL)	BI / ANALISTA (Power BI / Tableau)	
APIs	→ Ingesta	→ Staging	→	Dashboards
Kafka	→ Limpieza	→ Modelos	→	Informes
Bases de datos	→ Pipeline ETL	→ Métricas	→	KPIs
	→ Warehouse	→ Documentación	→	Análisis
	DATA SCIENTIST (Python / notebooks)	ML ENGINEER (MLOps / producción)		
	Experimentación	→ Feature store		
	Modelado ML	→ Serving del modelo		
	Métricas del modelo	→ Monitorización		
		→ CI/CD para modelos		

Cada flecha entre roles es un punto potencial de fricción que, cuando no se gestiona, se convierte en un bucle que consume tiempo del equipo sin generar valor.

6.2 El bucle infinito: Data Engineer ↔ Analista de negocio / BI Developer



Por qué el Data Engineer dice "aquí tienes los datos, ya está"

No es arrogancia. Técnicamente, su trabajo terminó. El Data Engineer optimiza para que el pipeline no falle, los datos lleguen a tiempo, el schema sea correcto técnicamente y la infraestructura escale.

No optimiza para que el nombre `amt_net_rev_adj_fx` signifique algo, que los nulos en `fecha_entrega` tengan explicación de negocio, o que el analista de Sevilla y el de Madrid calculen el mismo KPI igual.

Por qué el analista dice "los datos están sucios"

Nombres incomprensibles. `cd_cli`, `tp_op`, `dt_cre`, `fl_actv`. El analista no sabe qué es nada sin preguntar.

Granularidad incorrecta. El analista quiere ventas por mes, pero la tabla tiene una fila por cada línea de cada pedido de cada ticket.

Métricas que no cuadran. La tabla de ventas dice 100.000 €. La tabla de clientes dice 97.000 €. Nadie documentó la diferencia.

Nulos sin explicación. El campo `canal_venta` tiene un 15% de nulos. ¿Significa venta directa? ¿Error del sistema? ¿Migración?

Duplicados silenciosos. Duplicados del pipeline que el Data Engineer conoce y filtra mentalmente, pero nunca documentó.

Las cinco fases del bucle

Fase 1 — El malentendido inicial. El analista construye el dashboard. Las cifras no cuadran con el Excel histórico. El analista asume que los datos están mal. El Data Engineer asume que el analista no sabe usarlos.

Fase 2 — Las reuniones de alineamiento. Se convocan reuniones para "alinear definiciones". Descubren que tienen definiciones distintas de cosas básicas: ¿"venta" incluye devoluciones? ¿El mes es la fecha del pedido o la de entrega?

Fase 3 — Los parches. El Data Engineer crea vistas específicas. Una para ventas, otra para finanzas, otra para operaciones. Cada vista incorpora reglas de negocio distintas sin documentar.

Fase 4 — La fragmentación. Seis meses después hay quince vistas calculando variantes de "ventas". Nadie da la misma cifra. En cada reunión alguien pregunta "¿cuál es la cifra buena?".

Fase 5 — La crisis de confianza. La dirección deja de confiar en los datos. Se toman decisiones por intuición. El equipo de datos pierde credibilidad.

💡 En la mayoría de las empresas, el 80% del trabajo de un analista es encontrar los datos, limpiarlos y entender qué significan. Solo el 20% es el análisis real que genera valor. El Analytics Engineer existe para invertir esa proporción.

6.3 El bucle del científico de datos: Data Engineer ↔ Data Scientist

Este bucle existe por las mismas razones que el anterior, pero es más complejo y con consecuencias más graves.

Las cuatro fricciones específicas

"Necesito los datos crudos, no los agregados." El Data Engineer pasó semanas construyendo modelos agregados limpios. El científico llega y dice que necesita los datos a nivel de evento individual porque las agregaciones eliminan la señal para entrenar su modelo.

```
Data Engineer: "Eso son datos crudos sin limpiar, hay duplicados y nulos raros"
Científico:    "Ya me encargo yo de filtrar, dame acceso directo"
               → Tres semanas después: cinco reuniones preguntando qué significan los campos
```

El feature engineering interminable. El científico necesita variables complejas que no existen en ninguna tabla. Cada feature requiere que el Data Engineer implemente joins y ventanas temporales en Spark. El científico no sabe exactamente qué features necesita hasta que experimenta.

```
Científico: "Necesito la feature X" → Data Engineer implementa (3 días)
Científico: "X no mejora, necesito Y" → Data Engineer implementa (3 días)
Científico: "Y tampoco, prueba con Z y una variante de X" → ...semanas
```

Los entornos no coinciden. El científico entrena en local con pandas sobre 100.000 filas. En producción hay que ejecutar el modelo sobre 500 millones de filas en Spark.

```
Científico: "El modelo funciona perfectamente, aquí está el notebook"
Data Engineer: "Esto es un notebook Jupyter con pandas.
               Yo necesito código Spark que corra en producción"
Científico: "Eso es vuestro problema, yo entrego el modelo"
```

Este problema tiene nombre: **el abismo entre experimentación y producción**, o *"works on my machine"*.

Los datos cambian y el modelo se rompe silenciosamente. El Data Engineer renombra una columna por razones técnicas legítimas. El modelo sigue corriendo en producción. Las predicciones no explotan con un error: simplemente se vuelven malas. Tres semanas después alguien nota que las recomendaciones son raras. Se investiga. Era el renombrado de columna. Esto se llama **data drift** (cambian los valores) y **schema drift** (cambia la estructura).

Las cinco fases del bucle

Fase 1 — El proyecto prometedor. La dirección aprueba un proyecto de ML. "Vamos a predecir la churn de clientes." El científico empieza con entusiasmo.

Fase 2 — El descubrimiento del estado real de los datos. El científico necesita el historial de los últimos dos años. Descubre que los primeros seis meses están en un sistema legacy, que el campo "fecha de baja" no se registró hasta hace un año, y que la definición de "cliente activo" cambió tres veces sin documentación.

Fase 3 — El modelo que no se puede poner en producción. El científico tiene buenas métricas en local. Pero su código es pandas, la librería que usó no tiene equivalente en Spark, y el Data Engineer tarda semanas en reproducir el pipeline de features.

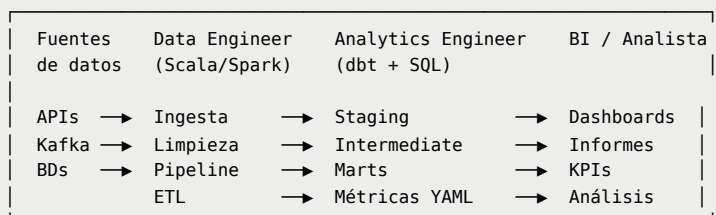
Fase 4 — El modelo en producción que nadie monitoriza. El modelo está en producción. Nadie es responsable de monitorizarlo. No hay alertas de degradación.

Fase 5 — "El ML no funcionó". La dirección concluye que el proyecto fue un fracaso. En realidad el modelo era bueno, pero el problema era organizativo e infraestructural.

6.4 El Analytics Engineer: el perfil que rompe el primer bucle

Por qué nació este rol

El Analytics Engineer nació para ser dueño de la capa que nadie reclamaba: los modelos de datos que convierten los datos crudos en métricas de negocio limpias y coherentes.



La herramienta central: dbt

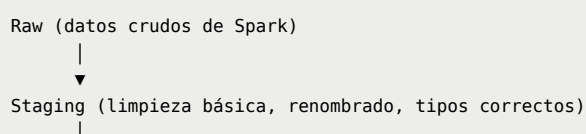
dbt (data build tool) organiza, versiona, testea y documenta las transformaciones SQL dentro del data warehouse.

Spark (Scala)	dbt (SQL)
Procesa datos a cualquier escala	Transforma datos ya en el warehouse
Código Scala / Python	SQL puro
Para volúmenes enormes	Para lógica de negocio y modelado
Corre en clúster distribuido	Corre dentro del warehouse

En muchas empresas **Spark y dbt coexisten**:

- Spark → mueve y procesa datos masivos hacia el warehouse
- dbt → transforma y modela esos datos dentro del warehouse

La arquitectura en capas



```

▼
Intermediate (joins, lógica de negocio intermedia)
|
▼
Marts (tablas finales listas para BI: fct_ventas, dim_clientes...)

```

Métricas canónicas en YAML

```

metrics:
- name: ventas_totales
  label: Ventas Totales
  model: ref('fct_ventas')
  measure: sum
  expr: importe
  dimensions:
    - ciudad
    - categoria
    - fecha

```

Una métrica, una verdad. Cualquier herramienta que lea la capa semántica usa esta definición.

Tests automáticos de calidad

```

models:
- name: fct_ventas
  columns:
    - name: id_venta
      tests:
        - unique
        - not_null
    - name: importe
      tests:
        - not_null
        - accepted_range:
            min_value: 0

```

Si algún test falla, el pipeline se detiene y el equipo recibe una alerta antes de que nadie vea datos incorrectos en producción.

Cómo rompe el bucle

- Escribe las definiciones **en código revisable en Git**, no en documentos Word
- Crea una capa de datos diseñada **para el análisis**, no para la ingestión
- Habla **los dos idiomas**: entiende al Data Engineer y al director financiero

Perfil del Analytics Engineer

Aspecto	Descripción
Herramienta principal	dbt, SQL avanzado, Git

Aspecto	Descripción
Lenguaje	SQL + YAML para métricas
Trabaja con	Datos ya en el warehouse
Output	Modelos limpios, métricas canónicas, tests, documentación
Conoce negocio	Mucho — es su diferenciador clave
Conoce sistemas	Medio
Posición en el equipo	Entre el Data Engineer y el analista, habla los dos idiomas

En España en 2025, el Analytics Engineer senior está en el mismo rango salarial que el Data Engineer senior, y en algunos casos por encima, porque el perfil es más escaso.

6.5 El ML Engineer / MLOps: el perfil que rompe el segundo bucle

Por qué nació este rol

Data Engineer	ML Engineer	Científico de datos
Pipeline de datos	Feature store Infraestructura de entrenamiento Serving del modelo Monitorización CI/CD para modelos	Experimentación Modelado Métricas del modelo

Qué hace el ML Engineer

Feature Store. Repositorio centralizado de features calculadas que tanto el entrenamiento como la inferencia en producción usan. Resuelve el problema de que el modelo se entrena con features calculadas de una forma y en producción se calculan de otra.

```
Sin Feature Store:
Entrenamiento: pandas → calcula "compras_ultimos_30_dias"
Producción:    Spark  → calcula "compras_ultimos_30_dias" (distinto)
Resultado:     el modelo no funciona igual que en entrenamiento

Con Feature Store:
Ambos leen del Feature Store → misma feature, mismo cálculo → consistencia
```

Gestión del ciclo de vida con MLflow:

```
Científico experimenta → MLflow registra métricas, parámetros, artefactos
                        → Se comparan versiones del modelo
                        → El mejor modelo se promueve a producción
                        → El modelo antiguo se archiva
```

Monitorización de modelos:

- **Data drift:** la distribución de los datos de entrada cambia respecto al entrenamiento
- **Concept drift:** la relación entre features y etiquetas cambia (el mundo cambia)
- **Model decay:** el rendimiento disminuye con el tiempo

Reentrenamiento automático. Cuando los datos se actualizan, el pipeline reentrena el modelo automáticamente, lo evalúa contra métricas predefinidas, y si supera al modelo anterior lo despliega sin intervención manual.

El stack habitual de ML Engineering

Experimentación: MLflow / Weights & Biases
 Feature Store: Feast / Hopsworks / Tecton
 Orquestación: Apache Airflow / Prefect / Kubeflow
 Serving: BentoML / Seldon / Spark batch
 Monitorización: Evidently AI / WhyLogs / Grafana
 Infraestructura: Kubernetes / Docker / Spark

Perfil del ML Engineer

Aspecto	Descripción
Herramienta principal	MLflow, Spark, Kubernetes, Airflow, Docker
Lenguaje	Python (principal) + Scala para componentes Spark
Trabaja con	Modelos ML + pipelines de datos + infraestructura cloud
Output	Modelos en producción, Feature Store, monitorización, CI/CD
Conoce ML	Medio-alto — no investiga algoritmos pero los entiende bien
Conoce sistemas	Mucho — es su punto fuerte

6.6 Tabla comparativa de todos los roles

	Data Engineer	Analytics Eng.	Data Scientist	ML Engineer
Herramienta principal	Spark, Kafka, Airflow, Scala	dbt, SQL, warehouse SQL	Python, pandas, scikit-learn, PySpark	MLflow, Feast, Spark, Kubernetes, Airflow
Output	Datos en el warehouse	Modelos limpios, métricas, tests	Modelos ML, análisis	Modelos en producción, Feature Store
Conoce negocio	Poco	Mucho	Medio	Poco-medio
Conoce sistemas	Mucho	Medio	Poco	Mucho
Fricción principal	Con todos los demás	Con Data Eng. y analistas	Con Data Eng. y ML Engineer	Con Data Sci. y Data Eng.

Herramientas de visualización por perfil

Perfil	Herramientas habituales
Data Engineer (Scala/Spark)	Grafana (métricas pipeline), Superset, Jupyter/Almond
Analytics Engineer	dbt docs (catálogo), Lightdash, Superset
BI Developer / Analista	Power BI, Tableau, Looker, Qlik Sense
Data Scientist	Jupyter, matplotlib, seaborn, plotly

Perfil	Herramientas habituales
ML Engineer	MLflow UI, Grafana (monitorización modelos), Evidently AI

6.7 Qué significa "datos legacy"

"Datos legacy" significa datos que viven en sistemas **antiguos, difíciles de modificar, mal documentados, pero que no se pueden apagar porque la empresa depende de ellos para funcionar hoy**.

La palabra clave no es "antiguo". Es **"no se puede apagar"**.

 **Analogía:** Es como la fontanería de un edificio de los años 60. Funciona. Nadie sabe exactamente cómo está distribuida por las paredes. Nadie tiene los planos originales. Cada vez que alguien intenta tocarla algo inesperado ocurre. Pero no puedes demoler el edificio para rehacerla porque la gente sigue viviendo dentro.

Por qué existen los sistemas legacy

- El tiempo pasa y la tecnología evoluciona (COBOL de 1985 razonable en 1985, problemático en 2025)
- Los que lo construyeron se fueron (el conocimiento vivía en tres personas)
- Se parcheó en lugar de reescribirse (capas y capas de parches acumulados)
- El coste de migrar es mayor que el de aguantar ("otro año más" lleva repitiéndose quince años)

Cómo se manifiesta para un Data Engineer con Spark

```
// Técnicamente esto funciona en segundos
val df = spark.read
  .format("jdbc")
  .option("url", "jdbc:oracle:thin:@servidor:1521:BDLEGACY")
  .option("dbtable", "CD_CLI_EST_OP")
  .load()

df.show()
// +-----+-----+-----+-----+-----+
// |CD_CLI|TP_EST|FL_A|CD_OP_OR|DT_ULT_MOV|
// +-----+-----+-----+-----+-----+
// | 00123|    A|   1|   0045| 20150312|

// ¿Qué es TP_EST = "A"? ¿FL_A = 1 significa activo?
// ¿DT_ULT_MOV es una fecha en formato YYYYMMDD o un número?
// Nadie lo sabe. Investigar puede llevar semanas.
```

Los sistemas legacy más comunes en España

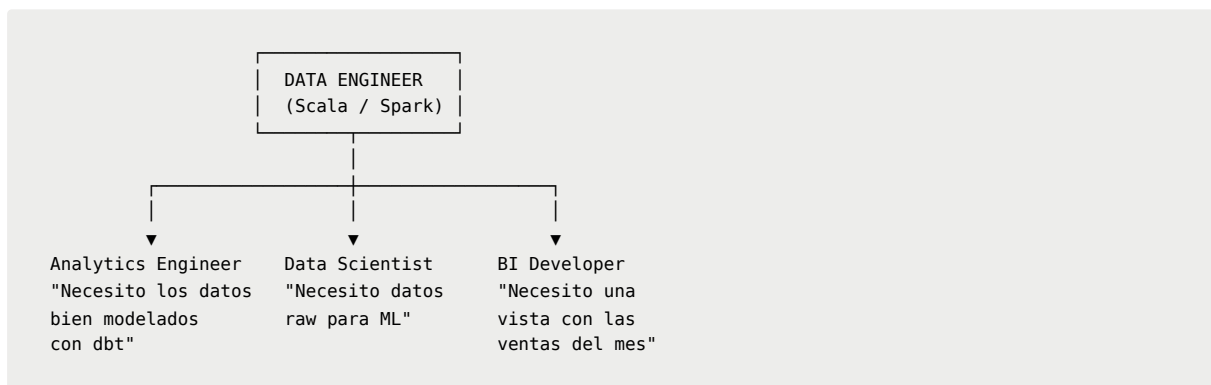
Sistema	Sectores	Problema principal
SAP ERP	Todas las empresas medianas/grandes	Personalización acumulada, conectores especializados

Sistema	Sectores	Problema principal
Oracle antiguo	Banca, seguros, telecomunicaciones, AAPP	Esquemas de los años 90 no modernizados
AS/400 (IBM iSeries)	Banca, retail, fabricación	Modelo de datos completamente diferente a lo moderno
Mainframe COBOL	Bancos grandes, aseguradoras	Proyectos de migración que llevan décadas sin terminar
Ficheros planos / Excel	Empresas de todos los tamaños	Legacy aunque el fichero se creara ayer

Por qué no se migra todo de una vez

- El sistema está procesando transacciones reales ahora mismo
- Nadie sabe exactamente qué hace el sistema actual (no hay especificaciones)
- El coste se ve claramente; el beneficio es difuso
- Los proyectos de migración tienen un historial de fracasos que genera aversión institucional

6.8 Lo que todo esto significa para el Data Engineer con Spark



La diferencia entre un Data Engineer junior y uno senior

El junior domina Spark, Scala, los conectores y la optimización de jobs. El senior hace todo eso **y además:**

- Documenta los cambios de schema antes de implementarlos y avisa a todos los consumidores
- Diseña los pipelines pensando en quién va a consumir los datos y cómo
- Entiende suficiente lógica de negocio para hacer preguntas inteligentes antes de construir
- Anticipa las fricciones organizativas antes de que ocurran
- Sabe cuándo el problema no es técnico sino de comunicación

En equipos de datos maduros, el trabajo técnico raramente es lo que más tiempo ocupa. Lo que más ocupa es la coordinación, la documentación, la alineación de definiciones y la gestión de dependencias entre sistemas y entre personas.

ANEXO — Caso real: Proyecto de predicción de churn (Telecomunicaciones)

 Este documento es un **anexo**. Ilustra con código concreto qué hace cada rol en un proyecto de ML real. Es material de referencia para entender cómo se articula el trabajo de un equipo de datos completo en producción.

Este anexo ilustra con código concreto exactamente qué hace cada rol en un proyecto de Machine Learning real. El caso es la predicción de abandono de clientes (churn): identificar qué clientes van a cancelar su contrato en los próximos 30 días para llamarles con una oferta de retención antes de que se vayan.

El mapa del proyecto completo

PROYECTO CHURN – QUIÉN HACE QUÉ		
DATA ENGINEER (Scala/Spark)	DATA SCIENTIST (PySpark/Python)	ML ENGINEER (MLOps/Infra)
Ingesta desde sistemas legacy	Análisis exploratorio del dataset	Validación del modelo antes de producción
Limpieza y tipado	Selección de features	Job inferencia batch Spark
Feature engineering a escala	Elección algoritmo	Feature Store (Feast)
VectorAssembler / StringIndexer	Ajuste hiperparámetros	Monitorización data drift
Dataset en Delta	Cross-validation	CI/CD del modelo
Preprocesador serializado	Umbral de decisión	Reentrenamiento automático (Airflow DAG)
	Interpretabilidad	
	Entrega en MLflow	
ANALYTICS ENGINEER (dbt + SQL)		
Modelo dbt fct_predicciones_churn · Métricas canónicas YAML		
Tests de calidad sobre predicciones · Vista resumen para Power BI		

PARTE 1 — Data Engineer (Scala / Spark)

Paso 1 — Ingesta desde múltiples fuentes

```
// Datos de contratos desde Oracle legacy
val contratos = spark.read
  .format("jdbc")
  .option("url", "jdbc:oracle:thin:@crm-prod:1521:CRMDB")
  .option("dbtable", "CD_CONTRATOS")
  .option("partitionColumn", "ID_CONTRATO")
  .option("lowerBound", "1")
  .option("upperBound", "100000000")
  .option("numPartitions", "20")
  .load()
```



```
// Datos de llamadas al soporte desde Kafka
val llamadas = spark.read
  .format("kafka")
  .option("kafka.bootstrap.servers", "kafka-prod:9092")
  .option("subscribe", "llamadas-soporte")
  .load()

// Datos de facturación desde S3
val facturas = spark.read
  .format("parquet")
  .load("s3://datalake-prod/facturas/")
```

Paso 2 — Limpieza y normalización

```
val contratosLimpios = contratos
  .withColumnRenamed("CD_CLI", "cliente_id")
  .withColumnRenamed("DT_ALT", "fecha_alta")
  .withColumnRenamed("TP_TAR", "tipo_tarifa")
  .withColumn("fecha_alta",
    to_date(col("fecha_alta").cast("string"), "yyyyMMdd"))
  .filter(col("cliente_id").isNotNull)
  .filter(col("FL_ACT") === 1)
  .dropDuplicates("cliente_id")
```

Paso 3 — Feature engineering a escala

El científico especificó qué variables necesita. El Data Engineer las implementa en Spark para millones de clientes — esto no lo implementa el científico:

```
import org.apache.spark.sql.expressions.Window

val ventana90dias = Window
  .partitionBy("cliente_id")
  .orderBy(col("fecha_llamada").cast("long"))
  .rangeBetween(-90 * 86400, 0)

val ventana3meses = Window
  .partitionBy("cliente_id")
  .orderBy(col("año_mes"))
  .rowsBetween(-2, 0)

val features = contratosLimpios
  .withColumn("antiguedad_dias",
    datediff(current_date(), col("fecha_alta")))
  .join(
    llamadas
    .withColumn("llamadas_90d", count("*").over(ventana90dias))
```

```

        .select("cliente_id", "llamadas_90d").distinct(),
        Seq("cliente_id"), "left"
    )
    .join(
        facturas
        .groupBy("cliente_id", "año_mes")
        .agg(sum("importe").alias("importe_mes"))
        .withColumn("importe_3m", sum("importe_mes").over(ventana3meses))
        .select("cliente_id", "importe_3m").distinct(),
        Seq("cliente_id"), "left"
    )
    .withColumn("tiene_incidencia_abierta",
        when(col("estado_incidencia") === "ABIERTA", 1).otherwise(0))
    .na.fill(0, Seq("llamadas_90d", "importe_3m"))

```

Paso 4 — Preparación del dataset con Spark MLlib

```

import org.apache.spark.ml.feature.{VectorAssembler, StringIndexer, StandardScaler}
import org.apache.spark.ml.Pipeline

val indexer = new StringIndexer()
    .setInputCol("tipo_tarifa")
    .setOutputCol("tipo_tarifa_idx")
    .setHandleInvalid("keep")

val assembler = new VectorAssembler()
    .setInputCols(Array(
        "antiguedad_dias", "llamadas_90d", "importe_3m",
        "tiene_incidencia_abierta", "tipo_tarifa_idx"
    ))
    .setOutputCol("features_raw")

val scaler = new StandardScaler()
    .setInputCol("features_raw")
    .setOutputCol("features")
    .setWithMean(true).setWithStd(true)

val pipelinePreprocesador = new Pipeline()
    .setStages(Array(indexer, assembler, scaler))

val preprocesador = pipelinePreprocesador.fit(features)

// Guardar el preprocesador serializado – se usará en producción
preprocesador.write.overwrite()
    .save("hdfs://modelos/preprocesador_churn_v2")

// Dataset final listo para ML

```

```

val datasetML = preprocesador.transform(features)
    .select("cliente_id", "features", "churn_label")

datasetML.write.format("delta").mode("overwrite")
    .save("hdfs://datasets/churn_dataset_v2")

println(s"✅ Dataset listo: ${datasetML.count()} clientes")

```

Hasta aquí llega el Data Engineer. El dataset está limpio, normalizado, en Delta Lake. El preprocesador está serializado.

PARTE 2 — Data Scientist (PySpark / Python)

El científico **no arranca desde los datos crudos**. Arranca desde el dataset que preparó el Data Engineer.

Análisis exploratorio

```

from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("churn-experimentacion").getOrCreate()

df = spark.read.format("delta").load("hdfs://datasets/churn_dataset_v2")
print(f"Dataset: {df.count()} clientes")

df_pandas = df.toPandas()

# Distribución de la variable objetivo
print(df_pandas["churn_label"].value_counts(normalize=True))
# 0    0.87 ← 87% no hace churn
# 1    0.13 ← 13% hace churn → dataset desbalanceado

```

Experimentación con múltiples algoritmos

```

from pyspark.ml.classification import LogisticRegression, GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
import mlflow

train, test = df.randomSplit([0.8, 0.2], seed=42)
evaluator = BinaryClassificationEvaluator(
    labelCol="churn_label", metricName="areaUnderROC"
)

# Experimento 1: Regresión logística (baseline)
with mlflow.start_run(run_name="logistic_regression_baseline"):
    lr = LogisticRegression(featuresCol="features", labelCol="churn_labe

```

```

l", maxIter=100)
auc_lr = evaluator.evaluate(lr.fit(train).transform(test))
mlflow.log_metric("auc", auc_lr)
print(f"Regresión logística – AUC: {auc_lr:.4f}")
# → AUC: 0.7234

# Experimento 2: GBT con búsqueda de hiperparámetros
with mlflow.start_run(run_name="gbt_tuned"):
    gbt = GBTClsifier(featuresCol="features", labelCol="churn_label")

    paramGrid = ParamGridBuilder() \
        .addGrid(gbt.maxDepth, [3, 5, 7]) \
        .addGrid(gbt.maxIter, [20, 50, 100]) \
        .addGrid(gbt.stepSize, [0.05, 0.1, 0.2]) \
        .build()

    cv = CrossValidator(estimator=gbt, estimatorParamMaps=paramGrid,
                        evaluator=evaluator, numFolds=5)
    modelo_gbt = cv.fit(train)
    auc_gbt = evaluator.evaluate(modelo_gbt.transform(test))
    mlflow.log_metric("auc", auc_gbt)
    print(f"GBT – AUC: {auc_gbt:.4f}")
    # → AUC: 0.8912

```

Ajuste del umbral de decisión

```

# Análisis del trade-off precision / recall con distintos umbrales
predicciones = modelo_gbt.transform(test)

for umbral in [0.3, 0.35, 0.4, 0.5]:
    preds = predicciones.withColumn(
        "pred", when(col("probability")[1] > umbral, 1.0).otherwise(0.0)
    )
    tp = preds.filter((col("pred")==1) & (col("churn_label")==1)).count()
    fp = preds.filter((col("pred")==1) & (col("churn_label")==0)).count()
    fn = preds.filter((col("pred")==0) & (col("churn_label")==1)).count()
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    print(f"Umbral {umbral} → Precision: {precision:.3f} Recall: {recall:.3f}")

# Umbral 0.30 → Precision: 0.541 Recall: 0.891
# Umbral 0.35 → Precision: 0.623 Recall: 0.847 ← elegido
# Umbral 0.40 → Precision: 0.712 Recall: 0.773
# Umbral 0.50 → Precision: 0.801 Recall: 0.641

```

Interpretabilidad del modelo

```
import pandas as pd

importancias = pd.DataFrame({
    "feature": ["tiene_incidencia_abierta", "llamadas_90d",
               "antiguedad_dias", "importe_3m", "tipo_tarifa_idx"],
    "importancia": modelo_gbt.bestModel.featureImportances.toArray()
}).sort_values("importancia", ascending=False)

print(importancias)
#               feature  importancia
# tiene_incidencia_abierta    0.3821 ← la más predictiva
#               llamadas_90d    0.2934
#               antiguedad_dias    0.1823

# Conclusión para el negocio:
# "Los clientes con incidencias abiertas sin resolver tienen 4 veces
#  más probabilidad de abandono. Resolverlas en < 48h reduciría
#  el churn estimado en un 23%."
```

Entrega documentada al ML Engineer

```
with mlflow.start_run(run_name="modelo_final_v3"):
    mlflow.spark.log_model(modelo_gbt.bestModel, "gbt_churn_model")
    mlflow.log_metric("auc_test", auc_gbt)
    mlflow.log_metric("precision_churn", 0.623)
    mlflow.log_metric("recall_churn", 0.847)
    mlflow.set_tag("estado", "listo_para_produccion")
    mlflow.set_tag("umbral_decision", "0.35")
    mlflow.set_tag("preprocesador", "hdfs://modelos/preprocesador_churn_v2")
```

PARTE 3 — ML Engineer (MLOps / Infraestructura)

Tarea 1 — Validación antes de promover a producción

```
import mlflow
from pyspark.ml.evaluation import BinaryClassificationEvaluator

modelo_candidato = mlflow.spark.load_model("models:/churn_predictor/Staging")
modelo_produccion = mlflow.spark.load_model("models:/churn_predictor/Production")

df_validacion = spark.read.format("delta") \
    .load("hdfs://datasets/churn_holdout_validacion")
```

```

evaluator = BinaryClassificationEvaluator(
    labelCol="churn_label", metricName="areaUnderROC"
)

auc_candidato = evaluator.evaluate(modelo_candidato.transform(df_validacion))
auc_produccion = evaluator.evaluate(modelo_produccion.transform(df_validacion))

# Solo se promueve si el candidato supera al modelo en producción
if auc_candidato > auc_produccion * 1.01:
    mlflow.transition_model_version_stage(
        name="churn_predictor", version=3, stage="Production"
    )
    print("✅ Modelo v3 promovido a producción")
else:
    print("⚠ El candidato no supera al modelo en producción. No se despliega.")

```

Tarea 2 — Job de inferencia batch en producción (Scala/Spark)

```

import org.apache.spark.ml.PipelineModel
import org.apache.spark.ml.classification.GBTClassificationModel

val preprocesador = PipelineModel.load("hdfs://modelos/preprocesador_churn_v2")
val modeloGBT = GBTClassificationModel.load(
    "mlflow-artifacts://modelos/gbt_churn_model/version/3"
)

val clientesHoy = spark.read.format("delta")
    .load("hdfs://datasets/clientes_activos_hoy")

val featuresHoy = preprocesador.transform(clientesHoy)
val prediccionesHoy = modeloGBT.transform(featuresHoy)

prediccionesHoy
    .filter(col("probability")(1) > 0.35)
    .select(
        col("cliente_id"),
        col("probability")(1).alias("prob_churn"),
        col("prediction"),
        current_date().alias("fecha_prediccion")
    )
    .orderBy(desc("prob_churn"))
    .write.format("delta").mode("overwrite")
    .save("hdfs://resultados/clientes_riesgo_churn_hoy")

```

```
println(s"✅ Predicciones escritas – equipo de retención notificado")
```

Este job se programa con **Airflow** para ejecutarse cada noche a las 2:00 AM.

Tarea 3 — Feature Store: consistencia entre entrenamiento y producción

```
from feast import FeatureStore, FeatureView, Field
from feast.types import Float64, Int64

churn_features = FeatureView(
    name="churn_cliente_features",
    entities=["cliente_id"],
    schema=[
        Field(name="antiguedad_dias", dtype=Int64),
        Field(name="llamadas_90d", dtype=Int64),
        Field(name="importe_3m", dtype=Float64),
        Field(name="tiene_incidencia_abierta", dtype=Int64),
    ],
    source=DeltaSource(
        path="hdfs://feature_store/churn_features",
        timestamp_field="fecha_calculo"
    ),
    ttl=timedelta(days=1)
)
# Tanto el entrenamiento como la inferencia leen del mismo Feature Store
# → Las features son idénticas en ambos contextos
```

Tarea 4 — Monitorización de data drift

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, ClassificationPreset

df_referencia = spark.read.format("delta") \
    .load("hdfs://datasets/churn_dataset_entrenamiento").toPandas()
df_produccion = spark.read.format("delta") \
    .load("hdfs://resultados/clientes_inferencia_ultima_semana").toPandas()

reporte = Report(metrics=[DataDriftPreset(), ClassificationPreset()])
reporte.run(reference_data=df_referencia, current_data=df_produccion)
reporte.save_html("hdfs://monitorización/drift_report_semana_21.html")

drift_detectado = reporte.as_dict()["metrics"][0]["result"]["dataset_drift"]

if drift_detectado:
    slack_client.chat_postMessage(
```

```

        channel="#alertas-ml",
        text="⚠ Data drift detectado en el modelo de churn. "
            "Considerar reentrenamiento."
    )

```

Tarea 5 — Reentrenamiento automático con Airflow

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

with DAG(
    dag_id="reentrenamiento_churn_mensual",
    schedule_interval="0 3 1 * *",    # primer día de cada mes a las 3:00
    AM
    start_date=datetime(2025, 1, 1),
    catchup=False
) as dag:

    actualizar_dataset = BashOperator(
        task_id="actualizar_dataset",
        bash_command="spark-submit hdfs://jobs/preparar_dataset_churn.scala"
    )

    reentrenar_modelo = BashOperator(
        task_id="reentrenar_modelo",
        bash_command="spark-submit hdfs://jobs/entrenar_churn.py "
            "--dataset hdfs://datasets/churn_dataset_v2"
    )

    validar_y_promover = PythonOperator(
        task_id="validar_y_promover",
        python_callable=validar_modelo_y_promover
    )

    notificar = PythonOperator(
        task_id="notificar_equipo",
        python_callable=lambda: slack_client.chat_postMessage(
            channel="#datos",
            text="✅ Modelo de churn reentrenado y desplegado automáticamente."
        )
    )

    actualizar_dataset >> reentrenar_modelo >> validar_y_promover >> notificar

```


PARTE 4 — Analytics Engineer (dbt + SQL)

El Analytics Engineer entra cuando las predicciones empiezan a generar resultados de negocio. Su trabajo es convertir esas predicciones en métricas comprensibles para que la dirección evalúe el ROI del proyecto.

Modelo dbt: tabla de resultados

```
-- models/marts/ml/fct_predicciones_churn.sql

WITH predicciones AS (
    SELECT cliente_id, prob_churn, churn_predicho, fecha_prediccion
    FROM {{ source('ml_resultados', 'clientes_riesgo_churn_hoy') }}
),
clientes AS (
    SELECT cliente_id, nombre, ciudad, tipo_tarifa,
        DATEDIFF(CURRENT_DATE(), fecha_alta) / 365.0 AS antiguedad_años
    FROM {{ ref('dim_clientes') }}
),
acciones_retencion AS (
    SELECT cliente_id, fecha_llamada, resultado_llamada, descuento_ofrecido
    FROM {{ source('crm', 'acciones_retencion') }}
),
real_churn AS (
    SELECT cliente_id, fecha_baja,
        CASE WHEN fecha_baja IS NOT NULL THEN 1 ELSE 0 END AS churn_real
    FROM {{ ref('dim_clientes') }}
    WHERE fecha_baja >= DATEADD(day, -60, CURRENT_DATE())
)

SELECT
    p.cliente_id, p.prob_churn, p.churn_predicho, p.fecha_prediccion,
    c.nombre, c.ciudad, c.tipo_tarifa, c.antiguedad_años,
    a.resultado_llamada, a.descuento_ofrecido,
    r.churn_real,
    CASE
        WHEN p.churn_predicho = 1 AND r.churn_real = 1 THEN 'verdadero_positivo'
        WHEN p.churn_predicho = 1 AND r.churn_real = 0 THEN 'falso_positivo'
        WHEN p.churn_predicho = 0 AND r.churn_real = 1 THEN 'falso_negativo'
        WHEN p.churn_predicho = 0 AND r.churn_real = 0 THEN 'verdadero_negativo'
    END AS resultado_prediccion
FROM predicciones p
LEFT JOIN clientes c ON p.cliente_id = c.cliente_id
```

```
LEFT JOIN acciones_retencion a ON p.cliente_id = a.cliente_id
LEFT JOIN real_churn r ON p.cliente_id = r.cliente_id
```

Métricas canónicas en YAML

```
metrics:
  - name: tasa_churn_mensual
    label: Tasa de Churn Mensual
    description: Porcentaje de clientes activos que cancelaron en el mes.
    model: ref('fct_predicciones_churn')
    measure: count_distinct
    expr: CASE WHEN churn_real = 1 THEN cliente_id END
    dimensions: [ciudad, tipo_tarifa, fecha_prediccion]

  - name: cobertura_modelo
    label: Cobertura del Modelo de Churn
    description: Porcentaje de churners reales que el modelo identificó (recall).
    model: ref('fct_predicciones_churn')
    measure: count_distinct
    expr: CASE WHEN resultado_prediccion = 'verdadero_positivo' THEN cliente_id END

  - name: efectividad_retencion
    label: Efectividad de Retención
    description: De los clientes en riesgo a los que llamamos, qué porcentaje aceptó.
    model: ref('fct_predicciones_churn')
    measure: count_distinct
    expr: CASE WHEN resultado_llamada = 'acepto_oferta' THEN cliente_id END

  - name: roi_retencion
    label: ROI del Programa de Retención
    description: Ingresos preservados menos coste de llamadas y descuentos.
    model: ref('fct_predicciones_churn')
    measure: sum
    expr: >
      CASE WHEN resultado_llamada = 'acepto_oferta'
        THEN importe_3m / 3.0 * 12 - descuento_ofrecido - 8.5
      END
```

Tests de calidad sobre las predicciones

```
models:
  - name: fct_predicciones_churn
```

```

columns:
  - name: cliente_id
    tests: [not_null, unique]
  - name: prob_churn
    tests:
      - not_null
      - accepted_range: {min_value: 0.0, max_value: 1.0}
  - name: resultado_prediccion
    tests:
      - accepted_values:
          values: [verdadero_positivo, falso_positivo,
                  falso_negativo, verdadero_negativo]
tests:
  - dbt_utils.expression_is_true:
      expression: "(SUM(churn_predicho) / COUNT(*)) < 0.30"
      name: tasa_churn_predicha_razonable

```

Vista para el dashboard de Power BI

```

-- models/marts/ml/rpt_efectividad_churn_mensual.sql

SELECT
  DATE_TRUNC('month', fecha_prediccion) AS mes,
  ciudad,
  tipo_tarifa,
  COUNT(DISTINCT cliente_id) AS clientes_evaluados,
  SUM(churn_predicho) AS clientes_en_riesgo,
  SUM(churn_real) AS churners_reales,
  SUM(CASE WHEN resultado_prediccion = 'verdadero_positivo' THEN 1 END) AS verdaderos_positivos,
  ROUND(
    SUM(CASE WHEN resultado_prediccion = 'verdadero_positivo' THEN 1 END)
    / NULLIF(SUM(churn_real), 0) * 100, 1) AS recall_pct,
  SUM(CASE WHEN resultado_llamada = 'acepto_oferta' THEN 1 END) AS clientes_retenidos,
  ROUND(
    SUM(CASE WHEN resultado_llamada = 'acepto_oferta' THEN 1 END)
    / NULLIF(SUM(CASE WHEN churn_predicho = 1
                     AND resultado_llamada IS NOT NULL THEN 1 END), 0)
    * 100, 1) AS tasa_retencion_pct,
  ROUND(SUM(

```

```

CASE WHEN resultado_llamada = 'acepto_oferta'
      THEN importe_3m / 3.0 * 12 - descuento_ofrecido - 8.5
END), 0)
AS roi_euros
FROM {{ ref('fct_predicciones_churn') }}
GROUP BY 1, 2, 3
ORDER BY 1 DESC, roi_euros DESC

```

Con esta tabla, el BI Developer abre Power BI, conecta al warehouse, arrastra `mes`, `tasa_retencion_pct` y `roi_euros` al lienzo y en diez minutos tiene el dashboard que la dirección necesita para evaluar si el proyecto de ML está generando valor.

El timeline completo del proyecto

DÍA 1-15: PREPARACIÓN

- Data Engineer (Scala) ingesta datos legacy + Kafka + S3
- Data Engineer construye feature engineering a escala en Spark
- Data Engineer prepara dataset ML y serializa preprocesador

DÍA 16-45: EXPERIMENTACIÓN

- Data Scientist (PySpark) hace EDA sobre el dataset preparado
- Data Scientist experimenta con LR, RF y GBT en MLflow
- Data Scientist ajusta hiperparámetros y umbral de decisión
- Data Scientist entrega modelo v3 documentado en MLflow

DÍA 46-60: PRODUCTIVIZACIÓN

- ML Engineer valida el modelo candidato contra el holdout
- ML Engineer implementa job de inferencia batch en Scala/Spark
- ML Engineer construye Feature Store con Feast sobre Delta Lake
- ML Engineer configura monitorización de drift con Evidently
- ML Engineer crea DAG de reentrenamiento automático en Airflow

DÍA 61-75: REPORTING

- Analytics Engineer crea modelo dbt `fct_predicciones_churn`
- Analytics Engineer define métricas canónicas en YAML
- Analytics Engineer escribe tests de calidad sobre las predicciones
- Analytics Engineer construye vista `rpt_efectividad_churn_mensual`
- BI Developer conecta Power BI y construye dashboard para dirección

DÍA 76+: OPERACIÓN CONTINUA

- Data Engineer: pipeline de datos se ejecuta cada noche
- ML Engineer: inferencia batch cada noche, reentrenamiento mensual
- ML Engineer: alertas si se detecta drift o degradación
- Analytics Engineer: los modelos dbt se actualizan solos
- BI Developer / Dirección: dashboard siempre actualizado